

第3章_运算方法与运算器_知识点整理

第3章 运算方法与运算器

关联：本章是计算机组成原理的核心章节，与第2章（数据信息的表示）紧密衔接——第2章介绍了数据的表示方法（原码、补码、浮点数等），本章则介绍如何对这些数据进行运算。第6章（中央处理器）中的ALU设计和数据通路与本章的运算器设计直接相关。

3.1 计算机中的运算

3.1.1 C语言中的位运算

位运算属于**数值运算**。

运算符	运算	说明
&	与	两个位都为1时结果为1
\	或	两个位至少一个为1时结果为1
~	非（取反）	0变1，1变0
^	异或	两位不同时结果为1

图片信息：PPT第5页，C语言位运算与AT&T汇编代码的对应关系（i=1, j=-1的与运算汇编代码）

3.1.2 C语言中的逻辑运算

逻辑运算属于**非数值运算**，所有参与运算的数以及运算结果只有0、1两个值。所有非0值都当做1处理。

运算符	运算
&&	与
\ \	或
!	非

图片信息：PPT第6页，C语言逻辑运算与汇编代码的对应关系

3.1.3 C语言中的移位运算

运算符	运算	说明
<<	左移	对应 逻辑左移
>>	右移	无符号数→ 逻辑右移 ；有符号数→ 算术右移

图片信息：PPT第7页，逻辑右移、算术右移、逻辑左移示意图

图片信息：PPT第7页，C语言移位运算与汇编代码的对应关系

移位运算的特点：

- **左移**：可能会产生溢出
 - `char x = 1111 1101 = -3`，逻辑左移后 `x = 1111 1010 = -6`（不溢出）
 - `unsigned char x = 1111 1101 = 253`，逻辑左移后 `x = 1111 1010 = 250`（正确结果506，溢出）
- **右移**：可能会产生误差
 - `char x = 1111 1101 = -3`，算术右移后 `x = 1111 1110 = -2`（正确结果-1.5，有误差）
 - `unsigned char x = 1111 1101 = 253`，逻辑右移后 `x = 0111 1110 = 126`（正确结果126.5，有误差）

桶型移位器：实现可变移位运算（ $x < y$ ， x 为32位变量， y 为5位变量， $y=0\sim 31$ ）。

图片信息：PPT第9页，Logisim实现的桶型移位器（左移0~31位）

3.1.4 C语言中的算术运算

运算符	运算
+	加
-	减
*	乘
/	除

汇编指令中的有符号数和无符号数运算：

运算	有符号数	无符号数
加减	<code>add</code> 、 <code>sub</code>	<code>add</code> 、 <code>sub</code> （与有符号数相同）
乘除	<code>imul</code> 、 <code>idiv</code>	<code>mul</code> 、 <code>div</code>

图片信息：PPT第11-12页，C语言算术运算与汇编代码的对应关系

3.2 定点加减法运算

3.2.1 补码加减法运算方法

1. 补码加法

$$[x + y]_{\text{补}} = [x]_{\text{补}} + [y]_{\text{补}} \pmod{M}$$

【例3.1】 $x = 0.1010, y = 0.0101$, 求 $[x + y]_{\text{补}}$

解: $[x]_{\text{补}} = 0.1010, [y]_{\text{补}} = 0.0101$

$$\begin{array}{r} [x]_{\text{补}} \quad 0.1010 \\ + [y]_{\text{补}} \quad 0.0101 \\ \hline [x + y]_{\text{补}} \quad 0.1111 \end{array}$$

【例3.2】 $x = -0.1010, y = -0.0100$, 求 $[x + y]_{\text{补}}$ 和 $x + y$

解: $[x]_{\text{补}} = 1.0110, [y]_{\text{补}} = 1.1100$

$$\begin{array}{r} [x]_{\text{补}} \quad 1.0110 \\ + [y]_{\text{补}} \quad 1.1100 \\ \hline [x + y]_{\text{补}} \quad 11.0010 \quad (\text{最高位1去掉}) \end{array}$$

$[x + y]_{\text{补}} = 1.0010$, 取反加1得 $x + y = -0.1110$

2. 补码减法

$$[x - y]_{\text{补}} = [x]_{\text{补}} - [y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}} \pmod{M}$$

关键: $[-y]_{\text{补}} = [y]_{\text{补}}$ 连同符号位取反 + 1

【例3.3】 $x = 0.1001, y = 0.0110$, 求 $[x - y]_{\text{补}}$

解: $[x]_{\text{补}} = 0.1001, [y]_{\text{补}} = 0.0110, [-y]_{\text{补}} = 1.1010$

$$\begin{array}{r} [x]_{\text{补}} \quad 0.1001 \\ + [-y]_{\text{补}} \quad 1.1010 \\ \hline [x - y]_{\text{补}} \quad 10.0011 \quad (\text{最高位1去掉, 模溢出}) \end{array}$$

$[x - y]_{\text{补}} = 0.0011$

【例3.4】 $x = -0.1001, y = -0.0110$, 求 $[x - y]_{\text{补}}$ 和 $x - y$

解: $[x]_{\text{补}} = 1.0111, [y]_{\text{补}} = 1.1010, [-y]_{\text{补}} = 0.0110$

$$\begin{array}{r} [x]_{\text{补}} \quad 1.0111 \\ + [-y]_{\text{补}} \quad 0.0110 \\ \hline [x - y]_{\text{补}} \quad 1.1101 \end{array}$$

$[x - y]_{\text{补}} = 1.1101$, 取反加1得 $x - y = -0.0011$

3.2.2 溢出及检测

1. 溢出的概念

溢出：两个数相加（相减）的结果超出了补码的表示范围。

【例3.5】 溢出示例

(1) 设 $[x]_{\text{补}} = 0.1011$, $[y]_{\text{补}} = 0.1100$, 求 $[x + y]_{\text{补}}$

$$\begin{array}{r} 0.1011 \\ +0.1100 \\ \hline 1.0111 \end{array} \quad (\text{两个正数相加结果为负数, 出错!})$$

原因: $x + y = 11/16 + 12/16 = 23/16 > 1$, 超出定点小数表示范围 $[-1, 1)$

(2) 设 $[x]_{\text{补}} = 1.0101$, $[y]_{\text{补}} = 1.0100$, 求 $[x + y]_{\text{补}}$

$$\begin{array}{r} 1.0101 \\ +1.0100 \\ \hline 10.1001 \end{array} \quad (\text{两个负数相加结果为正数, 出错!})$$

原因: $x + y = (-11/16) + (-12/16) = -23/16 < -1$, 超出表示范围

2. 溢出检测

三种常见的溢出检测方法:

方法一：根据操作数和运算结果的符号位是否一致

运算情况	是否溢出
正数 + 正数 = 正数	不溢出
正数 + 正数 = 负数	溢出
负数 + 负数 = 负数	不溢出
负数 + 负数 = 正数	溢出
正数 + 负数	不可能溢出

设 X_f 、 Y_f 为操作数的符号位, S_f 为运算结果的符号位:

$$V = \overline{X_f} \cdot \overline{Y_f} \cdot S_f + X_f \cdot Y_f \cdot \overline{S_f}$$

- $V = 0$: 没有溢出
- $V = 1$: 有溢出

【例3.6】 $x = -0.1011$, $y = 0.1100$, 求 $[x - y]_{\text{补}}$

解: $[x]_{\text{补}} = 1.0101$, $[y]_{\text{补}} = 0.1100$, $[-y]_{\text{补}} = 1.0100$

负数 - 正数 = 负数 + 负数, 结果为正数, 溢出了。

$$\begin{array}{r} 1.0101 \\ +1.0100 \\ \hline 10.1001 \end{array}$$

$$X_f = 1, Y_f = 1 \text{ (} [-y]_{\text{补}} \text{ 的符号位)}, S_f = 0$$

$$V = 1 \cdot 1 \cdot 1 + 0 \cdot 0 \cdot 0 = 1 \quad (\text{溢出})$$

方法二：根据最高数据位的进位与符号位的进位是否一致

$$V = C_f \oplus C_d$$

- C_f : 符号位产生的进位信号
- C_d : 最高有效位产生的进位信号

【例3.7】 $x = -111, y = 110$, 求 $[x - y]_{\text{补}}$

解: $[x]_{\text{补}} = 1,001, [y]_{\text{补}} = 0,110, [-y]_{\text{补}} = 1,010$

$$\begin{array}{r} 1,001 \\ +1,010 \\ \hline 10,011 \end{array}$$

$$C_f = 1, C_d = 0, V = 1 \oplus 0 = 1 \text{ (溢出, } -7 - 6 = -13 \text{ 超出 } -8 \sim +7 \text{ 范围)}$$

【例3.8】 $x = -011, y = 100$, 求 $[x - y]_{\text{补}}$

解: $[x]_{\text{补}} = 1,101, [y]_{\text{补}} = 0,100, [-y]_{\text{补}} = 1,100$

$$\begin{array}{r} 1,101 \\ +1,100 \\ \hline 11,001 \end{array}$$

$$C_f = 1, C_d = 1, V = 1 \oplus 1 = 0 \text{ (没有溢出, } -3 - 4 = -7 \text{ 在范围内)}$$

方法三：利用变形补码的符号位进行检测

变形补码（双符号位补码）：正数的符号位为00，负数的符号位为11。

- 运算结果符号位为 00 或 11：没有溢出
- 运算结果符号位为 01 或 10：产生溢出

$$V = S_{f1} \oplus S_{f2}$$

- S_{f1} 、 S_{f2} 分别表示运算结果的第1个符号位和第2个符号位
- S_{f1} 为结果真正符号位

【例3.9】 变形补码溢出检测

(1) 设 $[x]_{\text{补}} = 00.1011, [y]_{\text{补}} = 00.0111$, 求 $[x + y]_{\text{补}}$

00.1011

+00.0111

01.0010 （符号位01，正溢出）

$x + y = 11/16 + 7/16 = 18/16 > 1$

(2) 设 $[x]_{补} = 11.0101, [y]_{补} = 11.0011$, 求 $[x + y]_{补}$

11.0101

+11.0011

10.1000 （符号位10，负溢出）

$x + y = -11/16 - 13/16 = -24/16 < -1$

(3) 设 $[x]_{补} = 00.1011, [y]_{补} = 11.0011$, 求 $[x + y]_{补}$

00.1011

+11.0011

11.1110 （符号位11，没有溢出）

$x + y = 11/16 - 13/16 = -2/16$

图片信息：PPT第21页，溢出检测软件方法示意图

3.2.3 加减法的逻辑实现

1. 全加器（Full Adder, FA）

全加器真值表：

X_i	Y_i	C_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

逻辑表达式：

$$S_i = X_i \oplus Y_i \oplus C_i$$

$$C_{i+1} = X_i Y_i + (X_i \oplus Y_i) C_i$$

图片信息：PPT第23页，全加器逻辑电路实现（假设异或门延迟3T，与门和或门延迟1T）

2. 半加器 (Half Adder, HA)

$$S_i = X_i \oplus Y_i$$

$$C_{i+1} = X_i \cdot Y_i$$

图片信息：PPT第27页，半加器真值表和电路图

3. 多位串行加法器 (行波进位加法器)

由 n 个全加器构成 n 位串行加法器。

- 无符号加法溢出标志： $V = C_n$
- 有符号加法溢出标志： $V = C_n \oplus C_{n-1}$

图片信息：PPT第28页，图3.3 多位串行加法器

4. 可控加减法电路 (Controller Adder/Subtractor)

- $Sub = 0$ ：加法运算
- $Sub = 1$ ：减法运算 ($X - Y = X + /Y + 1$)

原理： $Y_i \oplus Sub = 1$ 相当于取反 (求反码)，结果 $+1$ 体现在 $C_0 = Sub = 1$ 。

- 无符号加法溢出： $V = C_n$
- 无符号减法溢出： $V = /C_n$
- 有符号加法/减法溢出： $V = C_n \oplus C_{n-1}$

图片信息：PPT第30页，图3.4 多位可控加减法电路

图片信息：PPT第31页，可控加减法溢出判断示意图

5. 先行进位加法器 (Carry Look-Ahead Adder, CLA)

进位生成函数： $G_i = X_i Y_i$

进位传递函数： $P_i = X_i \oplus Y_i$

$$C_{i+1} = G_i + P_i C_i$$

展开得：

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

4位先行进位电路：时间延迟为2T (1T与门 + 1T或门)

图片信息：PPT第34页，图3.6 4位进位生成/进位传递电路

图片信息：PPT第35页，图3.5 4位先行进位电路

成组进位函数：

$$G^* = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0$$

$$P^* = P_3P_2P_1P_0$$

$$C_4 = G^* + P^*C_0$$

图片信息：PPT第37页，图3.5 可级联的4位先行进位电路（CLA 74182）

4位先行进位加法器（快速加法器）：

- 得到 S_0 的时间延迟：6T（ P_0 生成3T + 异或门3T）
- 得到 S_1 、 S_2 、 S_3 的时间延迟：8T（再加2T进位电路）
- 得到 C_4 的时间延迟：5T

图片信息：PPT第38页，图3.6 4位先行进位加法器

16位加法器实现方式：

方式	结构	时间延迟
串行（图3.3）	—	$(2n + 4)T = 36T$
组内并行、组间串行（图3.7）	4个4位CLA串联	$14T$
组内并行、组间并行（图3.8）	4个4位CLA + 可级联先行进位电路	$12T$

图片信息：PPT第41页，图3.7 16位加法器（组内并行、组间串行）

图片信息：PPT第42页，图3.8 16位加法器（组内并行、组间并行）

3.3 定点乘法运算

3.3.1 原码一位乘法

1. 原码一位乘法运算方法

乘积符号： $P_0 = x_0 \oplus y_0$

乘积数值： $|P| = |x| \cdot |y|$

被乘数： $[x]_{\text{原}} = x_0.x_1x_2 \dots x_n$

乘数： $[y]_{\text{原}} = y_0.y_1y_2 \dots y_n$

乘积： $P = P_0.P_1P_2 \dots P_{2n}$

图片信息：PPT第46页，图3.9 原码一位乘法算法流程图

图片信息：PPT第46页，手工计算 $x = 0.1101, y = 0.1011$ 的乘法过程

【例3.10】 $x = 0.1101, y = -0.1011$, 用原码一位乘法求 $x \cdot y$

解:

乘积的符号 $P_0 = x_0 \oplus y_0 = 0 \oplus 1 = 1$

乘积的绝对值计算过程:

部分积	乘数 \$	y
00.0000	1011	$P = 0$
+ 00.1101		$y_n = 1, +\$$
00.1101	1011	
→ 00.0110	1101	逻辑右移一位
+ 00.1101		$y_n = 1, +\$$
01.0011	1101	
→ 00.1001	1110	逻辑右移一位
+ 00.0000		$y_n = 0, +0$
00.1001	1110	
→ 00.0100	1111	逻辑右移一位
+ 00.1101		$y_n = 1, +\$$
01.0001	1111	
→ 00.1000	1111	右移一位

乘积的绝对值 = 0.1000 1111

$$[x \cdot y]_{\text{原}} = 1.1000 \ 1111, \quad x \cdot y = -0.1000 \ 1111$$

2. 原码一位乘法的逻辑实现

图片信息: PPT第48页, 图3.10 原码一位乘法的逻辑实现

3. 原码二位乘法

每次完成二位的乘法, 可以减少一倍的乘法运算时间 (提高一倍的乘法运算速度)。

3.3.2 补码一位乘法 (Booth算法)

1. Booth算法公式推导

(1) 被乘数 x 符号任意, 乘数 y 符号为正数:

$$[x]_{\text{补}} \cdot [y]_{\text{补}} = [x \cdot y]_{\text{补}} = [x]_{\text{补}} \cdot y$$

(2) 被乘数 x 符号任意, 乘数 y 符号为负数:

$$[x]_{\text{补}} \cdot [y]_{\text{补}} = [x \cdot y]_{\text{补}} = [x]_{\text{补}} \cdot 0.y_1y_2 \dots y_n - [x]_{\text{补}}$$

(3) 被乘数 x 符号任意, 乘数 y 符号任意 (Booth算法):

$$[x]_{\text{补}} \cdot [y]_{\text{补}} = [x \cdot y]_{\text{补}} = [x]_{\text{补}} \cdot 0.y_1y_2 \dots y_n - [x]_{\text{补}} \cdot y_0$$

图片信息: PPT第52页, 图3.11 补码一位乘法算法流程图

2. 补码一位乘法运算

【例3.11】 $[x]_{\text{补}} = 1.0111, [y]_{\text{补}} = 1.0011$, 用补码一位乘法求 $[x \cdot y]_{\text{补}}$

解: $[-x]_{\text{补}} = 0.1001$ (取反加1)

部分积	乘数 y	y_{n+1}	说明
00.0000	10011	0	$P = 0$
+ 00.1001			$y_n y_{n+1} = 10, +[-x]_{\text{补}}$
00.1001	10011		
→ 00.0100	11001	1	算术右移一位
+ 00.0000			$y_n y_{n+1} = 11, +0$
00.0100	11001		
→ 00.0010	01100	1	算术右移一位
+ 11.0111			$y_n y_{n+1} = 01, +[x]_{\text{补}}$
11.1001	01100		
→ 11.1100	10110	0	算术右移一位
+ 00.0000			$y_n y_{n+1} = 00, +0$
11.1100	10110		
→ 11.1110	01011	0	算术右移一位
+ 00.1001			$y_n y_{n+1} = 10, +[-x]_{\text{补}}$
00.0111	0101		最后一步不移位

$$[x \cdot y]_{\text{补}} = 0.0111\ 0101$$

验证: $x = -0.1001, y = -0.1101, x \cdot y = 0.0111\ 0101$

3. 补码一位乘法的逻辑实现

图片信息: PPT第54页, 图3.12 补码一位乘法逻辑实现

3.3.3 阵列乘法器

基本原理: 采用手动乘法的方法, 用大量与门阵列同时产生各乘积项, 同时将大量一位全加器构成全加器阵列。

被乘数 $X = X_4X_3X_2X_1X_0$

乘数 $Y = Y_4Y_3Y_2Y_1Y_0$

乘积 $P = P_9P_8P_7P_6P_5P_4P_3P_2P_1P_0$

(无符号) 阵列乘法器的实现:

- 相加数产生电路: 1位乘法, 通过与门实现 (25个与门)
- 行内并行, 行间串行 (进位链)

图片信息: PPT第57页, 相加数产生电路

两种阵列乘法器结构:

结构	特点
横向进位	共 $n(n-1)$ 个全加器
斜向进位 (教材图3.13)	共 $n(n-1)$ 个全加器

图片信息: PPT第58页, 横向进位无符号阵列乘法器

图片信息: PPT第60页, 图3.13 斜向进位无符号阵列乘法器

3.3.4 补码阵列乘法器

原码阵列乘法器结构:

图片信息: PPT第63页, 原码阵列乘法器框图 ($n \times n$ 乘法阵列 + 相加数产生电路)

补码阵列乘法器:

图片信息: PPT第64页, 图3.14 补码阵列乘法器

条件	操作
$a_f = 1$	启动左边 n 位求补器, 将负数 a 转换为绝对值
$b_f = 1$	启动右边 n 位求补器, 将负数 b 转换为绝对值
$a_f = 0, b_f = 0$	两个求补器不工作, 直接送乘法阵列
$a_f \oplus b_f = 1$ (一正一负)	启动 $2n$ 位求补器, 结果为负数
$a_f = b_f$ (同号)	$2n$ 位求补器不工作, 结果为正数

3.3.5 乘法器性能优化

乘法器类型	时间复杂度	说明
原码一位乘法	$O(n^2)$	利用一个n位加法器进行n次累加
阵列乘法器	$O(n)$	利用 $n(n-1)$ 个全加器实现快速位积累加
Booth两位乘法	$O(\log_2 n)$	补码两位乘法，速度更快

流水线乘法器：

图片信息：PPT第66页，图3.15 乘法流水线

- 与门阵列 → 第1行FA → 第2行FA → 第3行FA → 第4行FA → 第5行ADD
- 每级之间用缓冲寄存器隔开，CLK时钟驱动

3.4 定点除法运算

3.4.1 原码一位除法

被除数： $[x]_{\text{原}} = X_0.X_1X_2\dots X_n$

除数： $[y]_{\text{原}} = y_0.y_1y_2\dots y_n$

商： $[Q]_{\text{原}} = q_0.q_1q_2\dots q_n$

余数： $R = 0.r_1r_2\dots r_n$

商的符号： $q_0 = x_0 \oplus y_0$

商的数值： $|Q| = |x| \div |y|$

要求： $|x| < |y|$ ，否则发生溢出（即 $|Q| < 1$ ）

图片信息：PPT第68页，手工除法运算示例（ $x = 0.1001001, y = 0.1011$ ）

1. 原码恢复余数法

【例3.12】 $[x]_{\text{原}} = 1.1001, [y]_{\text{原}} = 0.1011$ ，求 $[x \div y]_{\text{原}}$

解： $|x| = 0.1001, |y| = 0.1011, [-|y|]_{\text{补}} = 1.0101$

商的符号 $= 1 \oplus 0 = 1$

余数 R	商 Q	说明
00.1001	0.0000	初始余数 $R =$
+ 11.0101		减 y

余数 R	商 Q	说明
11.1110	0.0000	余数为负, 商0
+ 00.1011		加 \$
00.1001		
→ 01.0010	0.000	左移1位
+ 11.0101		减 \$
00.0111	0.0001	余数为正, 商1
→ 00.1110	0.001	左移1位
+ 11.0101		减 \$
00.0011	0.0011	余数为正, 商1
→ 00.0110	0.011	左移1位
+ 11.0101		减 \$
11.1011	0.0110	余数为负, 商0
+ 00.1011		加 \$
00.0110		
→ 00.1100	0.110	左移1位
+ 11.0101		减 \$
00.0001	0.1101	余数为正, 商1

商 $|Q| = 0.1101$, 余数 $|R| = 0.0000\ 0001$

$$[Q]_{\text{原}} = 1.1101, \quad [R]_{\text{原}} = 1.0000\ 0001$$

验证: $x = -9/16, y = 11/16, Q = -13/16, R = -1/256$

$$x = Q \cdot y + R = (-13/16) \cdot (11/16) + (-1/256) = -144/256 = -9/16$$

恢复余数法规则:

- 余数为负, 商0, 加 $|y|$ 恢复余数, 左移1位, 减 $|y|$
- 余数为正, 商1, 左移1位, 减 $|y|$

2. 原码不恢复余数法 (加减交替法)

【例3.13】 $[x]_{\text{原}} = 1.1001, [y]_{\text{原}} = 0.1011$, 求 $[x \div y]_{\text{原}}$

解: 运算结果中的 C 为进位标志, 商位与进位相同 $q = C$

C	余数 R	商 Q	说明
	00.1001	0.0000	初始余数 $R =$

C	余数 R	商 Q	说明
	+ 11.0101		减 \$
0	11.1110	0.0000	余数为负, 商0
	→ 11.1100	0.000	左移1位
	+ 00.1011		加 \$
1	00.0111	0.0001	余数为正, 商1
	→ 00.1110	0.001	左移1位
	+ 11.0101		减 \$
1	00.0011	0.0011	余数为正, 商1
	→ 00.0110	0.011	左移1位
	+ 11.0101		减 \$
0	11.1011	0.0110	余数为负, 商0
	→ 11.0110	0.110	左移1位
	+ 00.1011		加 \$
1	00.0001	0.1101	余数为正, 商1

商 $|Q| = 0.1101$, 余数 $|R| = 0.0000\ 0001$ (与例3.12相同)

不恢复余数法规则:

- 余数为正, 商1, 余数左移1位, **减**去除数
- 余数为负, 商0, 余数左移1位, **加**上除数
- 一会儿做加法, 一会儿做减法, 也称为**加减交替法**

3. 原码不恢复余数法的逻辑实现

图片信息: PPT第73页, 图3.16 原码不恢复余数法的逻辑实现

3.4.2 阵列除法器

1. CAS的结构及其工作原理

CAS (Controlled Adder Subtractor): 可控加减法单元

输入: X_i 、 Y_i 、 C_i 、 P

输出: S_i 、 C_{i+1}

- $P = 0$: CAS做加法运算
- $P = 1$: CAS做减法运算

$$S_i = X_i \oplus (Y_i \oplus P) \oplus C_i$$

图片信息：PPT第74页，CAS的结构和符号

2. 阵列除法器

图片信息：PPT第75页，图3.18 阵列除法器

3. 阵列除法器时间延迟分析

- n 位串行可控加减法电路时间延迟： $(2n + 7)T$
- n 位CAS阵列除法器需要 $(n + 1)^2$ 个CAS
- 阵列除法器时间延迟： $[2(n + 1) + 7]T \times (n + 1)$
- 时间复杂度： $O(n^2)$ （阵列乘法器为 $O(n)$ ）

3.5 浮点运算

3.5.1 浮点加减法运算

1. 阶码和尾数采用补码表示的浮点加减运算

运算步骤：

步骤	操作	说明
(1) 对阶	小阶向大阶对齐	阶码小的尾数右移，阶码加1
(2) 尾数运算	定点数的补码加减运算	
(3) 结果规格化	右规或左规	尾数绝对值 >1 则右规（右移1次，阶码+1）；尾数绝对值 <0.5 则左规（左移若干次，阶码-1）
(4) 舍入	末位恒置1法或0舍1入法	右规时尾数末尾被丢弃产生误差
(5) 溢出判断	阶码符号位为01或10时溢出	

【例3.14】 设 $x = 2^{-101} \cdot (-0.101011)$, $y = 2^{-010} \cdot (0.001110)$ ，阶码3位，尾数6位（均不含符号位），都用补码表示，计算 $x + y$

解：

$$[x]_{\text{补}} = 11\ 011, 11.010101 \quad [y]_{\text{补}} = 11\ 110, 00.001110$$

(1) 对阶： x （阶码=-5）向 y （阶码=-2）对齐， x 的尾数右移3位

对阶后 $[x]_{\text{补}} = 11\ 110, 11.111010(101)$

(2) 尾数运算:

$$\begin{array}{r} 11.111010(101) \\ +00.001110 \\ \hline 00.001000(101) \end{array}$$

(3) 规格化: 尾数 $00.001000(101)$ 为非规格化数 (绝对值太小), 左规2次

规格化后尾数 = $00.100010(1)$, 阶码减2变为 $11\ 100$

(4) 舍入: 0舍1入法, 尾数 = 00.100011

(5) 溢出判断: 阶码双符号位相同 (11), 没有溢出

$$[x + y]_{\text{补}} = 11\ 100, 00.100011, \quad x + y = 2^{-100} \cdot (0.100011)$$

【例3.15】 设 $x = 2^7 \cdot (25/32), y = 2^6 \cdot (-23/32)$, 阶码5位, 尾数7位 (含2位符号位), 计算 $x - y$ (0舍1入法)

解:

$$[x]_{\text{补}} = 00\ 111, 00.11001 \quad [-y]_{\text{补}} = 00\ 110, 00.10111$$

(1) 对阶: y (阶码=6) 向 x (阶码=7) 对齐, y 的尾数右移1位

对阶后 $[-y]_{\text{补}} = 00\ 111, 00.01011(1)$

(2) 尾数运算:

$$\begin{array}{r} 00.11001 \\ +00.01011(1) \\ \hline 01.00100(1) \end{array}$$

(3) 规格化: 尾数 $01.00100(1)$ 绝对值太大, 右规1次

规格化后尾数 = $00.10010(01)$, 阶码加1变为 $01\ 000$

(4) 舍入: 0舍1入法, 尾数 = 00.10010

(5) 溢出判断: 阶码双符号位为01, 发生溢出

$$[x - y]_{\text{补}} = 01\ 000, 00.10010 \quad (\text{溢出})$$

验证: $x = 100, y = -46, x - y = 146$ (超出 $-128 \sim +127$ 范围)

2. IEEE754浮点数的加减运算

IEEE754浮点数特点: 阶码用移码表示, 尾数用原码表示 (尾数 = $1.M$, 最高位隐藏)。

结果规格化:

- 尾数 = $1.xx \dots x$: 规格化尾数
- 尾数 = $1x.xx \dots x$: 绝对值 >2 , 需1次右规 (尾数右移1次, 阶码+1)
- 尾数 = $0.xx \dots x$: 绝对值 <1 , 需左规 (左移若干次直到 $1.xx \dots x$, 每左移1次阶码-1)

舍入处理: 就近舍入、朝 $+\infty$ 方向舍入、朝 $-\infty$ 方向舍入、朝0方向舍入

溢出判断:

- 右规时阶码为全1 (移码全1为最大正数): 规格化上溢
- 左规时阶码为全0 (移码全0为最小负数): 规格化下溢

【例3.16】 两个IEEE754单精度浮点数 $X = 00C0\ 0000H$, $Y = 0080\ 0000H$, 求 $X - Y$

解:

$$X = 0\ 00000001\ 1000 \dots 0 = 1.1 \times 2^{-126}$$

$$Y = 0\ 00000001\ 0000 \dots 0 = 1.0 \times 2^{-126}$$

(1) **对阶:** 阶码相同, 无需对阶

(2) **尾数相减:** $1.1 - 1.0 = 0.1$

(3) **规格化:** 0.1 为非规格化数, 左移1次, 尾数变为 1.0, 阶码减1变为 00000000 (全0)

(4) **舍入:** 无需舍入

(5) **溢出判断:** 阶码全0, 规格化下溢

IEEE754非规格化浮点数: $(-1)^S \times 0.M \times 2^{-126}$

$$X - Y = 0.1 \times 2^{-126} = 2^{-127} = 0040\ 0000H$$

3.5.2 浮点乘法运算

$$X = 2^m \times M_x, \quad Y = 2^n \times M_y$$

$$X \times Y = 2^{m+n} \times (M_x \times M_y)$$

运算步骤:

1. **阶码相加:** $m + n$
2. **尾数相乘:** $M_x \times M_y$
3. **规格化舍入:** 尾数相乘后若绝对值太小, 通过左规变为规格化数

3.5.3 浮点除法运算

$$X \div Y = 2^{m-n} \times (M_x \div M_y)$$

运算步骤:

1. **尾数调整:** 检查 $|M_x| < |M_y|$ 是否成立; 若不成立, 将 M_x 右移1次, 阶码 m 加1

2. 阶码求差: $m - n$
3. 尾数相除: $M_x \div M_y$
4. 规格化舍入: 若商的绝对值太小, 通过左规变为规格化数

3.6 运算器

3.6.1 定点运算器

1. 运算器组成

(1) 算术逻辑运算单元 (ALU)

ALU是对定点数据进行加工处理的**纯组合逻辑电路**。

信号	说明
a, b	参与运算的两个操作数
C_{in}	进位输入
result	运算结果
C_{out}	进位输出
operation	决定做什么运算
CF	Carry Out Flag, 加法进位/减法借位/左移溢出
ZF	Zero Flag, 运算结果为0则ZF=1
OF	Overflow Flag, 有符号数运算溢出则OF=1
SF	Sign Flag, 运算结果为负数则SF=1

图片信息: PPT第85页, 简单的1位ALU电路图

- (2) 通用寄存器组 (寄存器堆)
- (3) 输入、输出数据选择控制 (多路选择器MUX)
- (4) 内部总线 (IB: Internal Bus)

2. 运算器的基本结构

(1) 单总线结构运算器

图片信息: PPT第87页, 图3.20 单总线结构运算器

- LA、LB: 缓冲寄存器
- 执行 `ADD R0, R1` ($((R0) + (R1) \rightarrow R0)$) 需要3个时钟周期

(2) 双总线结构运算器

图片信息：PPT第88页，图3.21 双总线结构运算器

- IB1、IB2：内部总线1和2
- 通用寄存器堆有2个输出端、1个输入端
- 执行 `ADD R0, R1` 需要2个时钟周期

(3) 三总线结构运算器

图片信息：PPT第89页，图3.22 三总线结构运算器

- IB1、IB2、IB3：三条内部总线
- 执行 `ADD R0, R1` 只需要1个时钟周期

3. ALU设计

- 基本算术逻辑运算单元（1位ALU）
- 由 n 个1位ALU构成 n 位ALU

图片信息：PPT第90页，ALU设计示意图

3.6.2 浮点运算器

以浮点加减法为例，计算过程包括：对阶、尾数运算、结果规格化、舍入处理、溢出判断。

可以采用流水线的实现浮点加减法运算。

图片信息：PPT第91页，图3.26 浮点加减法流水线

本章小结

3.7.1 C语言运算与汇编对应

- 位运算（&、|、~、^）属于数值运算
- 逻辑运算（&&、||、!）属于非数值运算，非0值当1处理
- 移位运算（<<、>>），注意逻辑/算术移位的区别及溢出、误差
- 算术运算（+、-、*、/），有符号数和无符号数汇编指令不同

3.7.2 定点加减法运算

- 补码加法： $[x + y]_{\text{补}} = [x]_{\text{补}} + [y]_{\text{补}}$
- 补码减法： $[x - y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}}$
- 溢出检测三种方法：符号位判断、最高数据位进位判断、变形补码
- 全加器（FA）、半加器（HA）、串行加法器、可控加减法电路

- 先行进位加法器 (CLA): 进位生成函数 G 、进位传递函数 P
- 16位并行加法器两种实现: 组内并行组间串行、组内并行组间并行

3.7.3 定点乘法运算

- 原码一位乘法: 符号异或, 数值部分积移位累加
- 补码一位乘法 (Booth算法): 根据 $y_n y_{n+1}$ 判断 $+ [x]_{\text{补}} / + [-x]_{\text{补}} / + 0$
- 阵列乘法器: 与门阵列 + 全加器阵列, 时间复杂度 $O(n)$
- 补码阵列乘法器: 增加求补器处理符号
- 乘法器性能比较: 原码一位 $O(n^2) \rightarrow$ 阵列 $O(n) \rightarrow$ Booth两位 $O(\log_2 n)$

3.7.4 定点除法运算

- 原码恢复余数法: 余数为负则恢复余数再左移
- 原码不恢复余数法 (加减交替法): 余数为负直接左移加除数
- 阵列除法器: CAS单元构成, 时间复杂度 $O(n^2)$

3.7.5 浮点运算

- 浮点加减法五步: 对阶 $\rightarrow$ 尾数运算 $\rightarrow$ 规格化 $\rightarrow$ 舍入 $\rightarrow$ 溢出判断
- IEEE754浮点数运算: 阶码移码、尾数原码、隐藏最高位
- 浮点乘法: 阶码相加、尾数相乘、规格化舍入
- 浮点除法: 尾数调整、阶码求差、尾数相除、规格化舍入

3.7.6 运算器

- 状态标志: CF (进位)、ZF (零)、SF (符号)、OF (溢出)
- 单总线/双总线/三总线结构运算器: 分别需3/2/1个时钟周期
- 浮点运算器可采用流水线实现